

Microfrontend Migration Playbook

Stopniowa wymiana ekranów starej aplikacji bez big-bang rewrite

zespół platformy

2026-05-06

Architektura migracji legacy UI przez microfrontend embed

Dokument opisuje **architekturę** i **proces** stopniowej wymiany frontendu istniejącej aplikacji webowej. Założenie: aplikacja jest w starej wersji frameworka (np. Angular X.x, AngularJS 1.x, jQuery, ASP.NET WebForms) i ma być **przepisywana fragmentami**, ekran po ekranie, na nowy framework — bez przerwy w działaniu produkcyjnym.

Dokument jest **technologicznie neutralny** — nie wskazuje konkretnego frameworka. Decyzja o wyborze jest osobnym tematem.

1. Cel i ograniczenia

1.1. Cel biznesowy

- **Modernizacja UI** bez wstrzymywania rozwoju funkcjonalnego
- **Skrócenie time-to-market** dla nowych ekranów
- **Uspójnienie design systemu** (jeden zestaw komponentów, themingu, A11y)
- **Redukcja kosztu utrzymania** legacy stack (rzadsze kompatybilne biblioteki, mniej deweloperów)

1.2. Ograniczenia twarde

- Nie można **wstrzymać feature delivery** na czas migracji (12-24 miesiące)
- Backend pozostaje **bez zmian** w fazie migracji UI
- Auth/SSO/sesja użytkownika muszą działać **identycznie** dla obu wersji UI
- Migracja **odwracalna** — każdy zmigrowany ekran ma fallback do legacy
- Brak regresji wydajnościowej — nowy ekran $\leq$ czas legacy

1.3. Założenia o stanie wyjściowym

- Aplikacja monolityczna (jedno repo, jeden deploy)
 - Legacy stack: starszy framework SPA lub MPA z lekką “wyspowością”
 - 30-200 ekranów (CRUD + raporty + flowy procesowe)
 - Jednolity backend REST/GraphQL/WebSocket
 - Wewnętrzny zespół platformy + zespoły domenowe
-

2. Wybór technologii integracji

Trzy podejścia. Każde ma trade-offy.

2.1. iframe + postMessage (REKOMENDOWANE)

- **Izolacja:** pełna — dwie wersje frameworka nie kolidują (osobne window, osobne runtime)
- **Bezpieczeństwo:** same-origin lub kontrolowane cross-origin (CSP, X-Frame-Options)
- **Komunikacja:** tylko przez postMessage (typed contract)
- **Stylowanie:** theme propagowany przez init message; layout fixed (iframe size)
- **Routing:** lokalny w iframe, globalny w hoście; deep-link przez query params
- **Deploy:** nowy framework = osobna aplikacja, deploy niezależny od legacy
- **Wady:** iframe size management (auto-resize), modale “obcięte” do granic iframe (delegacja do hosta)

2.2. Web Components / Custom Elements

- **Bez iframe:** nowy komponent eksportowany jako tag HTML (<my-dashboard>)
- **Bardziej zintegrowany layout:** brak iframe granicy
- **Risk:** dwie wersje frameworka w tym samym window — możliwe konflikty (zone.js, polyfills, globalne style)
- **Bundle size:** każdy element = pełny runtime (kilkaset KB)
- **Wady:** niestabilność z różnymi wersjami runtime, wymaga ostrożnego config

2.3. Module Federation (Webpack 5) / native ESM federation

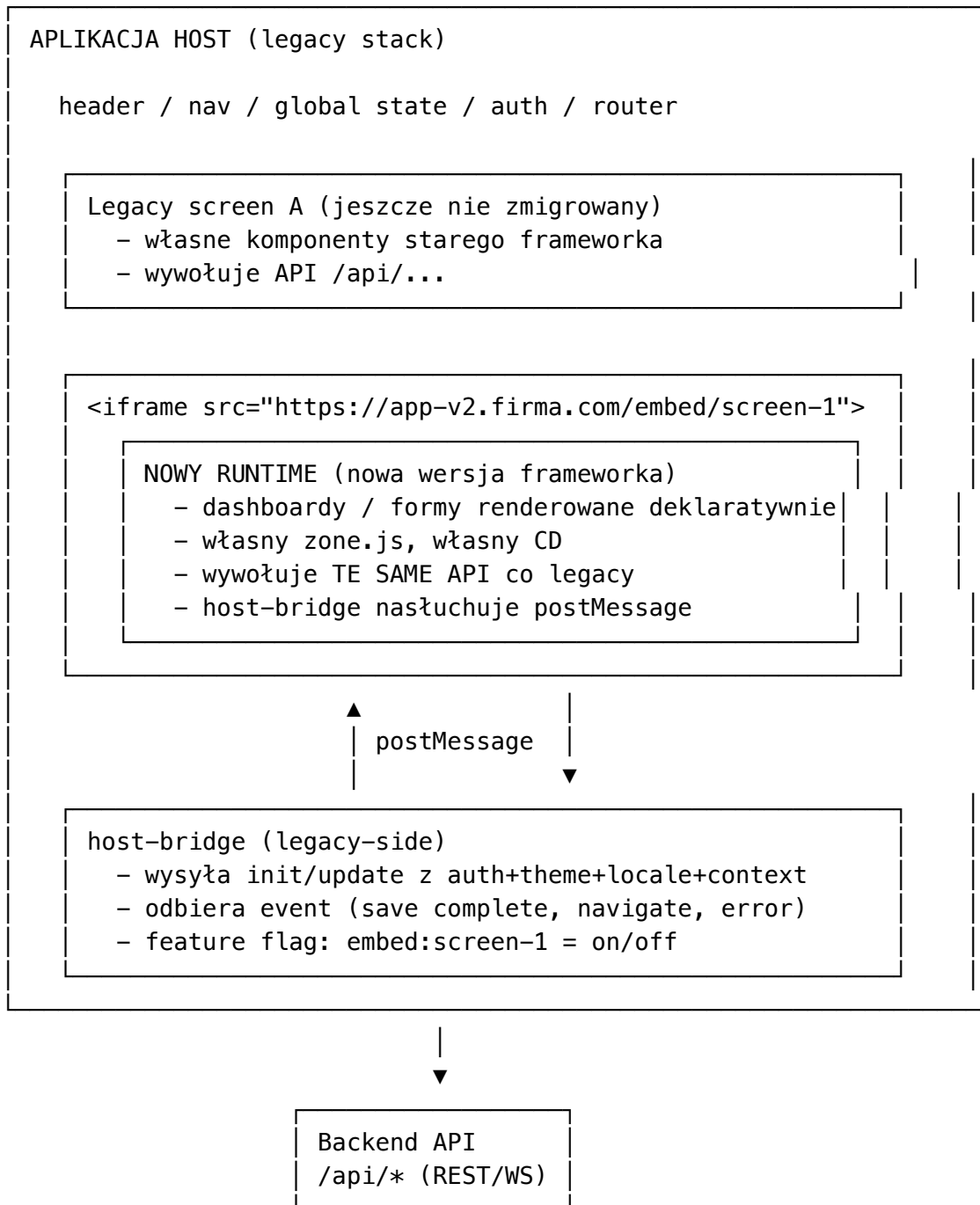
- **Współdzielenie kodu:** jedna instancja runtime, hosting modułów zdalnych
- **Wymaga:** identyczna lub kompatybilna wersja platformy po obu stronach
- **Niemożliwe** dla starej legacy + nowego frameworka w różnych majorach
- **Sens tylko:** gdy migrujesz **jednocześnie** legacy do nowej wersji

2.4. Decyzja

Dla migracji legacy ze **starym** frameworkiem na **nowy** frameworkiem (różne majory, różne runtime): **iframe + postMessage** jest jedynym podejściem dającym pełną izolację, predictability, niezależny deploy.

Web Components ma sens dla **mniejszych widgetów** (button library, charts, formularze pojedyncze) gdy oba runtime są kompatybilne. Module Federation — tylko gdy stack jest jednorodny.

3. Architektura wysokopoziomowa



Kluczowe punkty:

- **Stara aplikacja pozostaje shellem** — zarządza header, nav, globalnym stanem, autoryzacją, routerem
- **Każdy zmigrowany ekran** to iframe ładowany z URL nowego frameworka

- **Host-bridge** to cienka warstwa po stronie legacy (~3KB JS) która komunikuje się z iframe
 - **Backend nie zmienia się** — jeden endpoint, dwa konsumenci (legacy i nowy)
-

4. Setup po stronie nowego frameworka (embed-side)

Nowy framework jest **osobnym projektem** zbudowanym w nowym stack. Jej job:

- Wystawiać ekrany (dashboards/formularze/listy) jako self-contained widoki dostępne pod `/embed/<screenId>`
- Reagować na `postMessage` `init/update` od hosta (auth, theme, locale, params)
- Emitować `postMessage` `event` (save, navigate request, error)

4.1. Routing — `/embed/<screenId>`

Każdy ekran ma URL `embed-ready`. Iframe ładuje ten URL.

4.2. Bootstrap z host-bridge

Aplikacja inicjalizuje cienki bridge nasłuchujący `postMessage`:

```
// Pseudokod
initHostBridge({
  allowedOrigins: ['https://legacy.firma.com'],
  onInit: (screenId, data) => sessionStorage.set(data),
  onUpdate: (screenId, data) => sessionStorage.update(data),
  onCommand: (screenId, cmd) => commandHandler(cmd),
});
```

4.3. Wysyłanie eventów do hosta

Po akcjach użytkownika:

```
sendEventToParent('save-complete', { entityId: 'X' });
sendEventToParent('navigate-request', { route: '/screen/Y' });
sendEventToParent('error', { code: '...', message: '...' });
```

4.4. Auto-resize iframe

```
const ro = new ResizeObserver(([entry]) => {
  sendEventToParent('resize', { height: entry.contentRect.height });
});
ro.observe(document.body);
```

5. Setup po stronie hosta (legacy app)

5.1. Skrypt embed

W `index.html` legacy aplikacji:

```
<script src="https://app-v2.firma.com/embed.js"></script>
<script>
  EmbedRuntime.init({
    host: 'https://app-v2.firma.com',
    allowedOrigins: ['https://app-v2.firma.com'],
  });
</script>
```

5.2. Tag deklaracyjny (najprostsze)

```
<embed-screen
  screen="dashboard-1"
  data-context='{ "clientId": "CLI-001", "tenantId": "acme" }'
></embed-screen>
```

5.3. API imperatywne (gdy dane są dynamiczne)

```
const handle = EmbedRuntime.mount('#screen-container', {
  screenId: 'dashboard-1',
  data: {
    sessionToken: authService.getToken(),
    userId: user.id,
    role: user.role,
    theme: theme.current(),
    locale: i18n.current(),
    params: { clientId: 'CLI-001' },
  },
  onEvent: (event) => {
    switch (event.type) {
      case 'save-complete':
        list.refresh();
        break;
      case 'navigate-request':
        router.navigate(event.payload.route);
        break;
      case 'error':
        toast.error(event.payload.message);
        break;
    }
  },
});
```

// Aktualizacja sesji w trakcie (np. po refresh tokena):

```
handle.update({ sessionId: newToken });
```

```
// Unmount przy nawigacji:
handle.destroy();
```

5.4. Feature flag

Każdy zmigrowany ekran ma flagę. Stara wersja zostaje jako fallback.

```
if (featureFlags.isOn(`embed:${screenId}`)) {
  return renderEmbed();
} else {
  return renderLegacy();
}
```

6. Kontrakt komunikatów (TypeScript types)

Wspólny dla obu stron. Powinien żyć w **shared package** (@firma/embed-types) lub być wersjonowany jako contract API.

```
export type EmbedMessage =
  // Host → Embed
  | { type: 'embed:init'; sessionId: string; data: HostContextData }
  | { type: 'embed:update'; sessionId: string; data:
      Partial<HostContextData> }
  | { type: 'embed:command'; sessionId: string; command: HostCommand }
  // Embed → Host
  | { type: 'embed:ready'; sessionId: string }
  | { type: 'embed:event'; sessionId: string; event: string; payload:
      unknown }
  | { type: 'embed:resize'; sessionId: string; height: number }
  | { type: 'embed:error'; sessionId: string; code: string; message:
      string };
```

```
export interface HostContextData {
  readonly sessionId: string;
  readonly userId: string;
  readonly role: string;
  readonly theme: string;
  readonly locale: string;
  readonly params: Record<string, unknown>;
  readonly tenantId?: string;
  readonly featureFlags?: Record<string, boolean>;
}
```

```
export type HostCommand =
  | { kind: 'reload' }
  | { kind: 'focus'; targetId: string }
  | { kind: 'submit' }
  | { kind: 'cancel' };
```

Zasady kontraktu:

- Nieznany typ komunikatu → **ignorowany** (forward-compat)
- Dodawanie nowych typów = **niełamająca zmiana**
- Usuwanie / zmiana semantyki = **major** wersja kontraktu, sync deploy obu stron
- Wersja kontraktu obecna w `init.data.contractVersion` (negocjacja capability)

7. Auth, session, theme, locale

Element	Source of truth	Propagacja
Auth token	host (z SSO/auth service)	<code>init.data.sessionToken</code> na start, <code>update.data.sessionToken</code> przy refresh
User identity	host	<code>init.data.userId/role/tenantId</code>
Theme	host	<code>init.data.theme</code> (id z znanej palety); embed stosuje
Locale	host	<code>init.data.locale</code> ; embed konfiguruje i18n provider
Feature flags	host	<code>init.data.featureFlags</code> (subset dla embed)

Embed **nie czyta** `localStorage` hosta — single source of truth = `postMessage`.

Logout → host wysyła `command: { kind: 'reload' }` lub `iframe.src = ''`.

Refresh token expiracja → embed emituje event: `auth-expired` → host refreshuje token → wysyła `update.sessionToken`.

8. Routing i deep linking

8.1. URL hosta zachowuje stan ekranu

`https://legacy.firma.com/clients/CLI-001/dashboard`

↓

Decoduje: `clientId=CLI-001, tab=dashboard`

↓

Renderuje `<embed-screen screen="dashboard-1"`

`data-context='{ "clientId": "CLI-`

`001" }'>`

8.2. Embed routing wewnętrzny

W `iframe`, nowy framework ma własny router dla wewnętrznych tabs/wizard steps. Niezależny od hosta.

8.3. Globalna nawigacja

Embed nie nawiguje hosta bezpośrednio. Wysyła event: `navigate-request`:

```
sendEventToParent('navigate-request', { route: '/clients/CLI-002' });
```

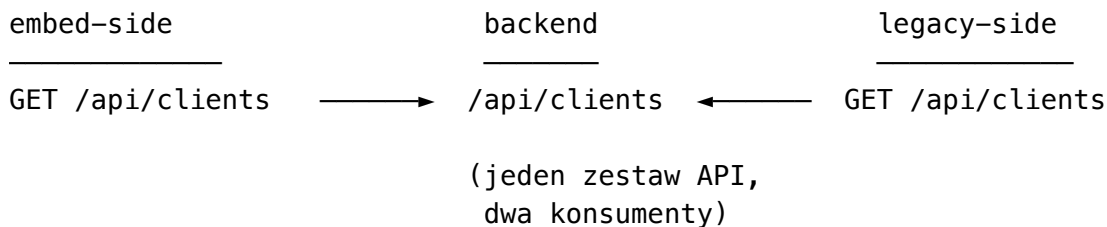
Host decyduje czy honoruje (np. odpalić inny embed, albo legacy screen).

8.4. Browser back button

Embed używa `history.replaceState` (nie `pushState`) — nie zaśmieca history hosta. Host kontroluje cofnięcie.

9. Datasource — wspólny backend

Nowy framework konsumuje **te same endpointy** co legacy:



Zaleta: zmiana backendu nie wymaga zmian w embed jeśli jest deklaratywna.

Auth: embed przekazuje `Authorization: Bearer ${session.sessionToken}` — backend nie odróżnia źródła.

CORS: backend musi zwracać `Access-Control-Allow-Origin: <embed-origin>` (lub `*` jeśli akceptowalne security-wise) i `Allow-Credentials: true` jeśli cookies używane.

10. Risk control

10.1. Feature flag per embed

Każdy embed = osobna flaga. Off → legacy fallback. On → embed.

10.2. Wersjonowanie bundle

Każda wersja deploymentu nowego frameworka ma własny URL: `/v1.2.3/...`. Stara wersja dostępna do natychmiastowego rollback.

```
EmbedRuntime.init({ host: 'https://app-v2.firma.com/v1.2.3' });
```


10.3. Canary rollout

Procent użytkowników (np. 5%) dostaje embed. Pozostali legacy. Stopniowo zwiększaj %.

```
const percent = featureFlags.percent('embed:screen-1');
const myBucket = hash(userId) % 100;
if (myBucket < percent) renderEmbed();
else renderLegacy();
```

10.4. Telemetry

Host loguje:

- Czas mount → ready (init → ready event)
- Liczba error events
- Liczba event events (proxy “user is doing things”)
- Network timing dla calls z embed

Threshold alarmów: error rate > 1%, time-to-ready > 3s p95.

10.5. Plan rollback

1. Feature flag → off (instant)
2. Embed bundle → poprzednia wersja (/v1.2.2/)
3. Backend nie zmieniany — bez ryzyka po stronie API

11. Pułapki (z doświadczenia)

#	Pułapka	Mitigacja
1	CSP frame-src w hoście blokuje iframe	Dodaj origin nowego frameworka do frame-src w meta CSP lub headers
2	X-Frame-Options nowy framework ustawia na DENY	Zmień na SAMEORIGIN lub usuń (CSP frame-ancestors zamiast)
3	Third-party cookies (cross-origin iframe) blokowane przez Chrome/Safari	Hosting embed w subdomenie legacy (app-v2.firma.com jako dziecko firma.com); session token przez postMessage nie cookies
4	CORS — embed pobiera /api/..., backend blokuje cross-origin	Backend zwraca Access-Control-Allow-Origin: <embed-origin> lub origin matchuje host (same-domain)

#	Pułapka	Mitigacja
5	zone.js / polyfill duplikacja w hoście vs embed	Iframe = osobny window = osobne polyfills. Brak konfliktu (problem tylko w Web Components)
6	localStorage shared między iframe a hostem (same-origin) — niebezpieczne dla auth	Embed NIE pisze do localStorage tokens — używa tylko in-memory state z postMessage
7	Layout broken — iframe ma fixed height, content w środku jest wyższy	embed: resize event → host aktualizuje iframe.style.height
8	Modale “obcięte” do granic iframe	Embed wysyła event: open-modal → host renderuje modal NA POZIO-MIE host
9	Drag & drop poza iframe nie działa cross-frame	Wewnątrz iframe OK; cross-frame DnD wymaga shared protocol (rzadko potrzebne)
10	Browser back button w iframe nie nawiguje hosta	Embed używa history.replaceState, emituje event: navigate-request
11	Auth refresh token wygasł — embed pokazuje 401	Host nasłuchuje event: auth-expired, refreshuje token, wysyła update.sessionToken
12	i18n niezsynchronizowane	Theme + locale ZAWSZE z init.data, nie z embed defaults
13	Print / Export w iframe — okno print obcięte	Embed wysyła event: print-request → host generuje print-version z full DOM
14	Keyboard shortcuts globalne kolizja (Ctrl+S)	Embed nie rejestruje globalnych shortcuts; host obsługuje, propaguje przez command

12. Plan migracji — 4 fazy

Faza 1 — Przygotowanie infrastruktury (1-2 sprinty)

- ☐ Backend: weryfikacja CORS + auth middleware dla nowego origin
- ☐ Hosting nowego frameworka: subdomena app-v2.firma.com, deploy pipeline
- ☐ Skeleton nowego frameworka: bootstrap, routing /embed/:screenId, host-bridge

- ☐ Host-side: `embed.js` w legacy, kontener `<embed-screen>` lub helper mount
- ☐ Kontrakt komunikatów: `typed in @firma/embed-types`, wersja 1.0
- ☐ Feature flag system (jeśli brak)
- ☐ Wybór 1 niskoryzykowny ekran read-only jako PoC

Faza 2 — Pierwszy embed end-to-end (1 sprint)

- ☐ Implementacja wybranego ekranu w nowego frameworka
- ☐ Iframe w legacy podmienia stary komponent
- ☐ `init/update/event` flow działa
- ☐ Theme + locale zsynchronizowane
- ☐ Feature flag on/off przełączna w runtime
- ☐ E2E test: parity functional z legacy
- ☐ Deploy → 5% użytkowników → 24h obserwacji → 50% → 100%

Faza 3 — Skalowanie (n sprintów, równolegle z developmentem)

- ☐ Tabela migracyjna: `screen, complexity, status, flag, pct, error-rate`
- ☐ Priorytety: read-only → CRUD → flowy procesowe
- ☐ Każdy ekran: implementacja → deploy → canary 5/50/100% → usunięcie kodu legacy
- ☐ Stara wersja kodu **usuwana** po 100% i 2 tygodniach bez rollbacku
- ☐ Telemetria każdego embed monitorowana ciągle

Faza 4 — Inwersja (cel końcowy)

- ☐ Wszystkie istotne ekrany w nowego frameworka
- ☐ Stara apka = tylko shell (auth + nav + globalny state)
- ☐ Refactor shellu na natywny shell nowego frameworka
- ☐ Wycofanie starej aplikacji

13. Wskaźniki sukcesu

Per embed:

- **Time to ready** (`init` → ready event) $\leq 1.5s$ p95
- **Error rate** (error events / sessions) $\leq 0.5\%$
- **Save success rate** $\geq 99\%$
- **Lighthouse score** > 80 (mobile)
- **A11y**: WCAG AA bez regresji vs legacy

Globalnie:

- **% ekranów zmigrowanych** (target po 12 miesiącach: 60-80%)
- **Średni czas migracji 1 ekranu** ≤ 1 sprint
- **NPS users** $>$ baseline
- **Bugfix-time** spadek (nowy framework łatwiejszy)
- **Build time** spadek po wycofaniu legacy

14. Kiedy zatrzymać migrację embed

- Aplikacja mała (< 5-10 ekranów) → big-bang rewrite jest tańszy
 - Backend wymaga jednoczesnej zmiany → najpierw migracja backendu
 - Aplikacja będzie wyłączona w < 12 miesięcy → koszt embed-infra się nie zwróci
 - Brak dostępu do legacy hostu (vendor) → tylko drop-in replacement
 - Performance budget krytyczny — iframe ma stały overhead (~50-100ms boot)
-

15. Checklisty

Setup hosta (legacy)

- ☐ CSP zawiera `frame-src <embed-origin>`
- ☐ `<script src="<embed-origin>/embed.js">` załadowane
- ☐ `EmbedRuntime.init()` wywołane raz przy bootstrap
- ☐ Helper `mountEmbed(slot, screenId, ctx)` wprowadzony do legacy DI
- ☐ Feature flag system gotowy
- ☐ Telemetry forward + monitoring dashboards

Setup embed (nowy framework)

- ☐ Bootstrap nowego frameworka
- ☐ `initHostBridge({ allowedOrigins, onInit, onUpdate, onCommand })`
- ☐ Routing `/embed/:screenId`
- ☐ Auth interceptor: wstrzykuje `sessionToken` z `$session` do każdego HTTP requesta
- ☐ Theme + locale stosowane z `init.data`
- ☐ Auto-resize: `ResizeObserver` → `sendEventToParent('resize', {height})`
- ☐ Error boundary: niezłapane błędy → `event: error`

Per-screen migracja

- ☐ Specyfikacja UX zwalidowana z PO/UX designerem
 - ☐ Theme parity z legacy
 - ☐ E2E test obejmuje all critical paths
 - ☐ Feature flag stworzony
 - ☐ Rollback procedure udokumentowana
 - ☐ Owner zespołu zatwierdza canary release
-

16. Pytania otwarte (do decyzji per projekt)

1. **Hosting:** subdomena (`app-v2.firma.com`) vs path (`firma.com/v2/`) — subdomena = czystsze CORS, path = brak third-party cookies issue
2. **Bundle distribution:** jeden deployment dla całej firmy czy per legacy app (multi-tenant)?
3. **Versioning:** semver per bundle (`v1.2.3`) czy major-only (`v1/v2`)?
4. **A11y:** czy nowy framework jest zgodna z naszym a11y baseline (WCAG AA)?

5. **Print/Export**: czy embed musi generować PDFy/CSVy w iframe, czy delegować do hosta?
6. **Multi-monitor / pop-out**: czy ekrany muszą być pop-out-able do osobnego okna? Wtedy iframe staje się trudniejszy
7. **Offline / PWA**: czy nowy framework musi działać offline? Service worker + cache embedded JSON?
8. **Mobile**: czy ten sam embed dla mobile? Iframe ma ograniczenia w mobile WebView

17. Anti-patterns — czego NIE robić

- **Wstrzykiwanie kodu nowego frameworka do legacy** (np. import jako npm package i mount jako Angular component) — kolizje runtime, niemożliwe debug
- **Współdzielony localStorage** dla auth — embed dostaje token przez postMessage, nie czyta z storage
- **Bezpośrednie wywołania funkcji** między host i embed — wszystko przez postMessage (typed contract)
- **Iframe srcdoc=** zamiast **src=** — utrudnia caching, network timing, debug
- **window.parent.someGlobal()** z embed — łamie izolację, blokowane przez cross-origin
- **Logowanie state hosta w embed** dla “convenience” — duplikacja, bug-prone
- **Inline style override** z hosta na zawartość embed (cross-frame) — niemożliwe; theme = postMessage

18. Decyzje, które trzeba podjąć przed startem

Decyzja	Opcje	Rekomendacja
Granica integracji	iframe / Web Component / Module Federation	iframe (dla różnych majorów)
Hosting	subdomena / path	subdomena
Bundle versioning	semver / major-only	semver z URL versioning
Auth mechanism	postMessage token / shared cookie	postMessage token
Feature flags	własny / Unleash / LaunchDarkly	dowolny — kluczowe że per-embed flag
Telemetry	własna / Datadog / Grafana	dowolna — kluczowe że per-embed metrics
Kontrakt komunikatów	shared TS package / Open-API / proto	shared TS (najprostsze, type-safe)
Modal strategy	embed renderuje / host renderuje	host renderuje (nie ograniczone iframe)

19. Roadmap kontraktu

Wersjonowanie kontraktu komunikatów oddzielnie od wersji nowego frameworka.

Contract v1.0 (today)

- └─ init/update/command/ready/event/resize/error
- └─ HostContextData: token/user/role/theme/locale/params

Contract v1.1 (Q3)

- └─ + event: 'auth-expired' (handshake refresh)
- └─ + command: {kind: 'print'}
- └─ + HostContextData.featureFlags

Contract v2.0 (rok+)

- └─ breaking: HostContextData.user → HostContextData.identity
- └─ deprecation cycle: 6 miesięcy

Embed i host negocjują wersję przez `init.data.contractVersion`. Embed odrzuca init z niekompatybilnym major.

20. Ryzyko organizacyjne

- **“Tylko jeden ekran” syndrome** — zespół przepisuje 1 ekran przez 6 miesięcy, polish zamiast rollout. Mitigacja: timebox per migracja (1 sprint maks)
 - **Owner conflict** — kto utrzymuje embed: zespół platformy czy domain? Mitigacja: platforma utrzymuje infra, domain utrzymuje screens
 - **Inversion of priorities** — feature delivery wstrzymuje migrację. Mitigacja: 30% capacity zespołu zarezerwowane na migrację
 - **Brak ownership po Faza 4** — stary stack wycofany ale kompetencji do refactor shellu brak. Mitigacja: planuj Fazę 4 z 6-mies. wyprzedzeniem
-

— koniec dokumentu